Assignment 2: Scalability of Support Vector Machines

Logan Nunno

February 2026

1 Task 1

The data structures I used for this part were numpy array for the weights and the x input features. This was used to make doing dot products and any other linear algebra operations faster because numpy is highly optimized. I also used a list to keep track of the losses of the losses during each of the epochs. The algorithms that were used were Stochastic Gradient Descent (SGD), Hinge loss, and L2 Regularization. The margin between the sample and its possible label is calculated. It does this for every sample by multiplying the true label and the dot product of the label and the weight plus the bias. This value given is used for the hinge loss function. the hinge loss function checks if the points is correctly classified and outside of the margin. If it is outside of the margin then it updates the weights by shrinking. If it is inside or the weight is wrong then it updates weights to both shrink the vector and move the decision boundary to better classify the current point. We use L2 Regularization to prevent overfitting of the data by penalizing the squared magnitude of the weights. This makes it so we try and maximize the margin to the hyperplane without overfitting the data.

The functions that were implemented in the class were init which will initialize the basic values of the LinearSVC. It can be given the learning rate, number of iterations, a random state, and a regularization parameter for use in L2 regularization. We have have a fit function that will be given the set of features and the true labels. This function will train the model based on the provided set of data. The net input function computes the preactivation value for a given input and the last is the predict function that takes a given input and makes a prediction for the given sample.

2 Task 2

Figure 1 is the provided demo of the make classification function. This function makes a random decision plane and uses that to make data points on both

sides of the boundary. The figure makes a data set with values of $d = 2, n = 100,$ and $u = 10$

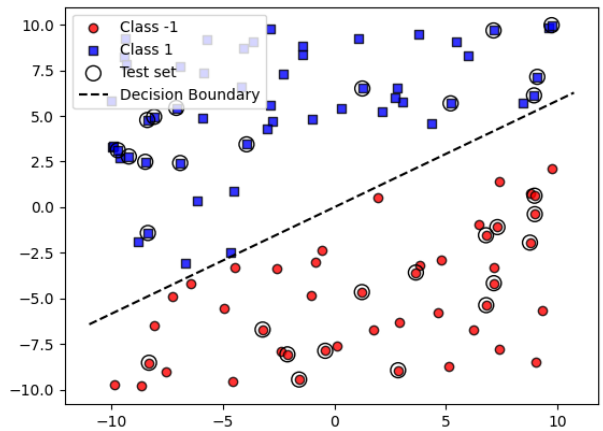

Figure 1: Make Classification Demo, d=2 n=100

The figure shows the two different classes of data $(-1, 1)$. It will also make the data into two different sets of data. The first is the training set that is 70% of the data these points are represent as the normal points of each of the classes. The second is the test set where the data is marked with circles around the points. The decision boundary shown in the figure as a dotted line represents the vector a that was used to generate the classes for the uniformly distributed data points.

3 Task 3

Figure 2 shows the loss convergences of 9 data sets created and tested using the custom made Linear SVM made in task 1. The data sets were made using the tool from task 2 and d was either $[10, 50, 100]$ and n was $[100, 500, 5000]$ and the u was 100 for all of the different sets. The SVM was tested with a learning rate of 0.00001, 1000 iterations, and a C value of 1. The loss convergence of the test with the data set is show in figure 2.

There is a lot of interesting things that can be seen from these loss convergence graphs. The way I am going to compare them is first by the number of samples and then see how the changing dimensions of the data effects it. In the first column we have 100 samples and we can see how the dimension of the data effects how fast the convergence happens. When $D = 10$ we see that the loss convergence is relatively slow compared to when $D = 50$ or 100. It takes more

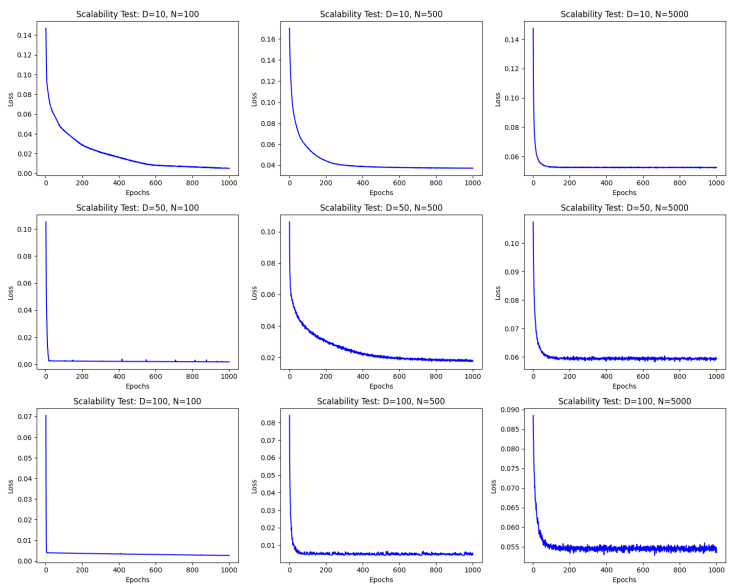

Figure 2: Custom Linear SVM Loss Convergence

than 1000 iterations to reach a plateau, and when $D = 50$ it takes less than 200 iterations, and the fastest is when D=100 it happens almost instantly. When we increase the number of samples to 500 we see this trend continue. We see that when $D = 10$ we do not reach a plateau by the time we have 1000 iterations and we are at a higher loss than the other graphs. When $D = 50$ we are still decreasing the slope so we would need more iterations to see the whole value of the loss but we can see that it is better than 10 and worse than 100. When we have $D = 100$ we see that again the drop is almost instant and we reach a plateau at a zero loss. When we increase the number of samples to 5000 we see that that both $D = 10$ and 50 never get close to zero as they plateau at a loss of 0.06 but with $D = 100$ we are still decreasing at 1000 iterations but starting to plateau.

The observation that can be seen with is that when we increase the dimensions (meaning increase the features) that the model is able to find the decision boundary much more easily but each of the individual operations is more expense because of the increased vector size. The reason that increasing the number of dimensions works in this case is because the data is perfectly linearly separable but in other cases where it is not this would be a bad thing and make it slower. We also see as we increase N the learning rate the the curve becomes noisier. When testing the code the I needed to decrease the learning rate by a lot in order for the curve to become smoother so it was less effected by the changes. We see a the opposite thing when we increase the size of number

of samples. When we have a small value of n (like 100) the model can be over fitted quickly leading to the flat line we see in some graphs. When n is large there are more constraints and edge cases. Since the learning rate is constant the small changes needed correctly satisfy the 5000 points of data causing the noise to increase. Another thing we see is that when the number of samples is very low and the number of dimensions is high we see that the model drops to 0 very fast and then is flat for the rest of the time. This means that with a low number of samples the number of iterations can be lowered by a lot in order to save time.

Table 1: Time Cost (Seconds) across Dimensions and Samples

Samples (n)	**Dimensions** (d)		
	10	**50**	**100**
100	0.5250	0.4417	0.4472
500	2.6192	2.4862	2.3241
5000	25.6043	35.3818	25.1191

The time cost for each of the data sets is given in table 1. We can see that the time scales linear with an increased number of samples. When we increase the number of samples by 5X we see time increase by 5X and when we increase it by 10X the time increase by 10X. Increasing the number of dimensions does not effect the time cost as much as the number of samples does. This is because we are using numpy operations and they are highly optimized and the increased number of features does not increase the time complexity as much as I originally thought. We can see from the graphs in figure 2 that when we increase d the the loss convergence lowers faster so in reality would could stop when we to a low loss and save time by increasing the dimensions.

4 Task 4

In Figure 3 and table 2 are the loss convergence and the time cost for dual linearSVM. In Figure 4 and table 3 are the loss convergence and the time cost for the primal linearSVM.

Table 2: Time Cost (Seconds) For Dual

Samples (n)	**Dimensions** (d)		
	10	**50**	**100**
100	0.0014	0.0028	0.0022
500	0.0019	0.0069	0.0191
5000	0.0070	0.0354	0.1189

The Dual option for linearSVM generally performs better when the number of samples is larger than the number of dimensions. However, we can see as we increase the number of dimensions the time cost of it increase significantly. We this when d goes from 10 to 100 for $n = 5000$ the time goes from 0.007 seconds to 0.1189 seconds.

Table 3: Time Cost (Seconds) For Primal

Samples (n)	**Dimensions** (d)		
	10	**50**	**100**
100	0.0013	0.0013	0.0014
500	0.0012	0.0077	0.0067
5000	0.0041	0.0788	0.3906

The Primal option is often faster for smaller datasets or when the number of features is small. We can see that Primal does not work as well for large datasets and large number of features in the exam where $n = 5000$ and $d = 100$. The primal version takes 0.3906 seconds and that is nearly 3.5 times slower than the dual problem.

According to the scikit-learn documentation, it is recommended to solve the dual problem when $n < d$ and the primal problem when $d < n$. This follows what we see in the time examples.

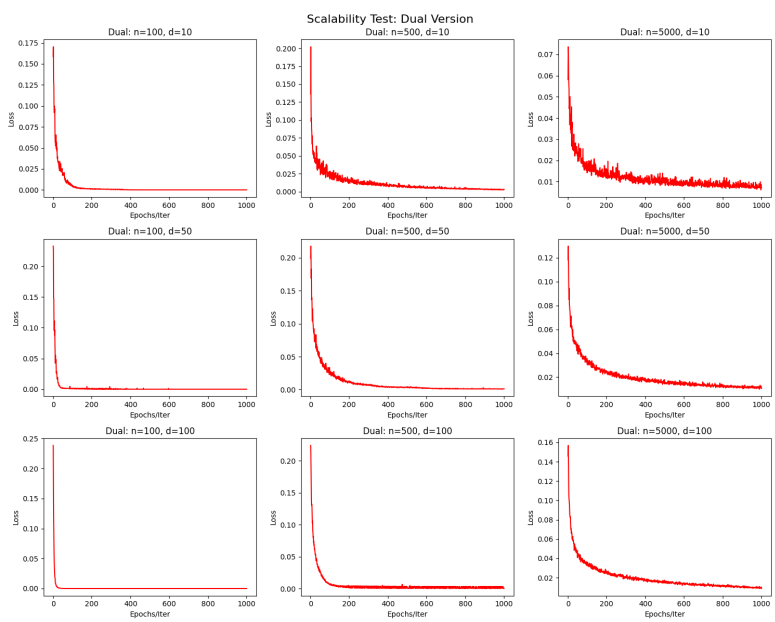

Figure 3: Dual Loss Convergence

Looking at both sets of graphs we can see that they both converge very quickly and they also drop to near zero in all versions of the data. When we compare the stability of the two versions primal shown in figure 4 has very smooth curves. They drop at the very start and then stay flat at basically zero. The dual graphs shown in figure 3 have more noise during the initial drop and we can see this very clearly when the number of samples increases. All of the graphs still converge in the end but they are less smooth and less stable than primal.

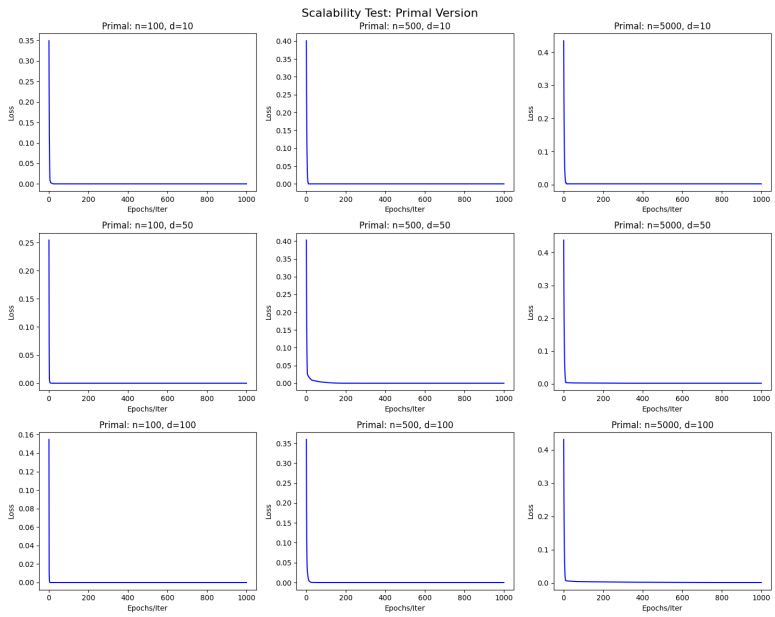

Figure 4: Primal Loss Convergence

The reason why the graphs for each of these look so different is because of the definition of the primal and dual problem and how they are different. With the primal problem we are focusing on the weights and the bias directly. We know that the graphs in figure 4 are very smooth. This is because the primal problem is very good at handling convex optimization problems. When we change the weights slightly the loss changes predictably. Primal depends on how many dimensions we have for how complicated the task is going to be. If we only have a few features then it is able to find the optimal line very quickly and not change from that place.

The dual problem is different from primal. It assigns a variable to every data point and tries to find which of the points are the support vectors. We know that in figure 3 that for almost all of the data sets we have a lot of visible noise in the graph. This happens because the algorithm for dual will update the variable for each data point one at a time causing there to be some noise in

the loss. As the algorithm goes through each of the different points, changing one point might slight throw off the balance of the other points. This makes the graph have the up and down lines in the convergence line before we each a minimum value. The complexity depends on the number of samples. That is why in table 2 dual starts to struggle as the number of samples increases.